Borla

Testing Report

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Testing_Report.pdf · Version 1.0 · 14 August 2026

1. Test strategy

Four layers, in the order they were actually run during the build:

1. **Unit tests** (Vitest) — pure functions with no I/O: OTP hashing/verification, the quiet-hours time-window predicate.
2. **Integration tests** (Vitest + Supertest) — the real Express app (`createApp()`) against a real PostgreSQL+PostGIS instance (a local Docker container running the exact same `postgis/postgis:16-3.4` image family Render uses), covering every route's happy path and its most important failure mode.
3. **Component tests** (Vitest + React Testing Library) — client-side rendering/interaction logic in isolation, with `fetch` mocked.
4. **System / User Acceptance Testing** — manual, scripted, end-to-end walkthroughs against the running application (server + Postgres + browser), exercising both roles and the admin console exactly as a real user would.

Security and usability checks (§5, §6) were folded into the same passes rather than run as a separate phase, which is appropriate at this scale.

2. Automated test results

2.1 Server (`npm run test -w server`)

```
✓ test/integration/requests.test.ts (11 tests)
✓ test/integration/reviews.test.ts (7 tests)
✓ test/integration/auth.test.ts (13 tests)
✓ test/integration/admin.test.ts (5 tests)
✓ test/integration/broadcasts.test.ts (4 tests)
✓ test/unit/redisRateLimit.test.ts (3 tests)
✓ test/unit/quietHours.test.ts (4 tests)
✓ test/unit/phone.test.ts (4 tests)
✓ test/unit/otp.test.ts (3 tests)
✓ test/unit/sms.test.ts (3 tests)

 Test Files  10 passed (10)
      Tests  57 passed (57)
   Duration  ~76s (real Postgres+Redis, no mocks)
```

`redisRateLimit.test.ts` and `phone.test.ts` are new since the Redis/BullMQ migration (Technical_Debt_Plan.md TD-05) and the phone-normalisation fix (D-07); `broadcasts.test.ts` now polls for the async BullMQ `fanout` job's side effect (a `broadcast_notifications` row) instead of asserting on a field the old synchronous handler used to return directly. `requests.test.ts` grew again for the button-triggered, server-verified arrival confirmation (FR-29/FR-29a, TD-17) and again for the one-live-request-per-collector guard and recency-ordered History (FR-36/FR-37, T-79/T-80) — the latter addition legitimately tripped the app-wide rate limiter mid-file purely from added request volume, fixed as D-23 (test infra, not a product defect); `reviews.test.ts` for review-in-request-context (FR-31) and the rating-aggregate staleness fix (D-11).

2.2 Client (`npm run test -w client`)

```
✓ src/pages/StatusChip.test.tsx (3 tests)

 Test Files  1 passed (1)
      Tests  3 passed (3)
```

Total: 60/60 automated tests passing at time of submission (client dropped from 5 to 3 — `ReviewForm.tsx` and its test were retired along with the double-blind review model, TD-15). Re-run with `npm test` from the repository root (requires a reachable Postgres+PostGIS and Redis — see `README.md`).

3. Test case log

Representative cases spanning all four rubric areas (functional, unit, integration, system/UAT). "Actual result" reflects the real run, including seven real defects directly caught by an inline test-case row (T-08, T-19, T-39, T-48, T-55, T-57, T-60 — the full set of thirteen defects, several found other ways, is in §4).

#	Test case	Layer	Expected result	Actual result
T-01	Request an OTP for a brand-new phone number with no `role`	Integration	`400` — role required to sign up	`400` returned with actionable message
T-02	Request an OTP, verify with the correct code	Integration	New user created, `access` / `refresh` issued	User created with `role='household'` , tokens returned
T-03	Verify with an incorrect 6-digit code	Integration	`400` , attempt counter incremented	`400` returned
T-04	Re-use an already-consumed OTP code	Integration	`400` — rejected as already used	`400` returned
T-05	`GET /auth/me` with no bearer token	Integration	`401`	`401` returned
T-06	`GET /auth/me` with a valid token	Integration	`200` with role-specific profile	`200` , `profile.alert_radius_m` present for household
T-07	Admin login with a non-admin phone number	Integration	`401`	`401` returned
T-08	**(defect)** OTP-verify signup path for a brand-new number	Integration (found while writing T-02)	New user row created with the requested role	**First implementation threw `role_missing` inside a dead t branch and never created the user** — see §4

#	Test case	Layer	Expected result	Actual result
T-09	Collector attempts to go online before admin verification	Integration	`403`	`403` returned
T-10	Verified + online collector, household broadcasts nearby	Integration	Fan-out notifies ≥1 collector; pin appears in that collector's `/pins/nearby`	`notifiedCollectors: 1` ; pin present with correct `distance_m`
T-11	Household creates a second broadcast while one is active	Integration	`409`	`409` returned
T-12	Collector role attempts to create a broadcast	Integration	`403` (role guard)	`403` returned
T-13	Household requests an **offline** collector	Integration	`409`	`409` returned
T-14	Full request lifecycle: requested → seen → accepted, contact reveal timing	Integration	Phone numbers absent pre-accept, present post-accept	Confirmed via direct field assertion pre/post
T-15	Reject a request **after** it was already accepted	Integration	`409` , status stays `accepted` (not resurrected to `rejected`)	`409` returned, status unchanged
T-16	A collector who wasn't the request's target tries to accept it	Integration	`409` (conditional update matches 0 rows)	`409` returned
T-17	Review submitted against a request that is **not yet accepted**	Integration	`400`	`400` returned
T-18	Duplicate review on the same accepted request	Integration	`409` (unique constraint)	`409` returned
T-19	**(defect)** Reply to a review before it is moderation-visible	Integration (found while writing this test)	`400` — cannot reply to a non-visible review	**First implementation only checked** `subject_id` **, not** `stat` **reply on a still-pending review** — see §4
T-20	*(superseded by T-58 — TD-15 removed the one-reply-per-review cap this test was checking)* Second reply attempt on an already-replied review	Integration	`409` (one reply per review)	`409` returned, at the time
T-21	Admin verifies a collector; collector can now go online; action appears in audit log	Integration	`200` → `200` ; `audit_log` contains `verify_collector`	All three confirmed
T-22	Admin suspends a household; suspended user's `/auth/me` is blocked; reinstate restores access	Integration	`403` while suspended, `200` after reinstatement	Confirmed
T-23	Admin edits an unknown config key	Integration	`404`	`404` returned
T-24	Full **system walkthrough**: household broadcast → collector sees pin on map → household clears it	System/UAT (manual, live server)	Pin visible then gone from collector's feed	Verified against the running app with real curl/browser sessions
T-25	Full **system walkthrough**: direct request → accept → contact reveal → review both sides → double-blind release fires within the 2-minute sweep window	System/UAT (manual, live server)	Both reviews become visible **simultaneously**, rating aggregates recompute correctly	`rating_avg` / `rating_count` confirmed via direct DB query after
T-26	Manual moderation fallback (no `GEMINI_API_KEY` set)	System/UAT	Submitted reviews appear in `/admin/moderation/queue → awaitingManual` , not silently published	Confirmed — both test reviews appeared, required explicit admin a
T-27	Quiet-hours predicate across a midnight-wrapping window	Unit	Inside/outside window classified correctly at boundaries	4/4 boundary cases correct
T-28	OTP hash/verify round-trip and mismatch rejection	Unit	Correct code verifies true, wrong code false	Both confirmed
T-29	`StatusChip` renders the correct label for every request status + an unknown fallback	Component	Exact label text present for each of 5 statuses + fallback	Confirmed
T-30	`ReviewForm` treats a `409` (already reviewed) the same as success	Component	Renders the thank-you state, not an error banner	Confirmed
T-31	Production client build (`vite build`) and server build (`tsc`) both compile clean	Build/System	Zero TypeScript errors, bundle produced	Both confirmed (`tsc --noEmit` clean; `dist/` produced)

#	Test case	Layer	Expected result	Actual result
T-32	GiantSMS phone-number normalisation (`+233…` / `233…` → local `0…` form)	Unit	Both prefixes normalise to the same local number; an already-local number is untouched	3/3 cases correct
T-33	GiantSMS end-to-end request against the real funded account, production	System (live)	Gateway accepts the send request	`POST /api/auth/otp/request` against `https://borla.onrende` `{"delivered":true}` — GiantSMS accepted the request at the H handset receipt not visually confirmed (test number is a placehold phone)
T-34	Unified sign-in: a new phone number gets an OTP with no role needed upfront; the code screen then requires role+name	Integration + scripted screenshot	`isNewUser:true` ; role/name fields appear only after the code step, submit creates the account	Confirmed both via `auth.test.ts` and a real headless-browser w
T-35	Unified sign-in: an existing (non-admin) number logs straight in from the code screen, no role/name prompt	Integration + scripted screenshot	`isNewUser:false` ; verify returns tokens with no extra fields required	Confirmed
T-36	Unified sign-in: an admin's phone number is auto-detected and routed to a password prompt instead of an OTP	Integration + scripted screenshot	`requiresPassword:true` , no OTP issued; UI shows the password form directly	Confirmed both ways
T-37	PWA service worker registers and activates on the production build	System (headless browser against `dist/`)	`navigator.serviceWorker.getRegistrations()` returns an active registration scoped to `/`	Confirmed: `{"scope":"http://localhost:5175/","active":true,"script` `<link rel="manifest">` present in the DOM
T-38	Seeded demo phone numbers always get the fixed `DEMO_OTP` , are exempt from the rate limit, and never trigger a real SMS attempt (D-06)	Integration	8 rapid `/otp/request` calls all succeed and return `devOtp:"482913"` , `delivered:false`	Confirmed via `auth.test.ts` ; also re-confirmed directly against
T-39	**(defect)** An already-registered phone number typed without the leading `+` (e.g. `233200000001`)	Integration + manual (found by the user against the live app, D-07)	Recognised as the same existing account, logs straight in	**First implementation stored/looked up the raw string, so** `+233200000001` / `233200000001` / `0200000001` **were three dif** — see §4
T-40	`normalizePhone` collapses all three Ghanaian phone-number input forms (`+233…` , `233…` , `0…`) to one canonical string	Unit	All three forms produce an identical result	4/4 cases correct (`phone.test.ts`)
T-41	Redis-backed presence: a collector going online is visible to a nearby household via `GEOSEARCH` , not a Postgres scan	Integration + manual (real local Redis)	`ZCARD geo:collectors` and `EXISTS presence:{id}` both reflect the online collector immediately	Confirmed via direct Redis inspection alongside the `/collectors`
T-42	Redis-backed per-collector notification rate limit caps fan-out messages independently per collector	Unit	Requests beyond the cap are rejected for that collector only; a different collector is unaffected	3/3 cases correct (`redisRateLimit.test.ts`)
T-43	Redis-backed OTP request rate limit caps requests per phone number	Unit	The (cap+1)th request for the same phone is rejected	Confirmed (`redisRateLimit.test.ts`)
T-44	Broadcast fan-out runs as an async BullMQ job, not inline in the request handler	Integration	`POST /broadcasts` returns before fan-out completes; a `broadcast_notifications` row appears shortly after, once the `fanout` job runs	Confirmed via `waitFor()` polling in `broadcasts.test.ts` — ro the poll window, HTTP response contains no notification count
T-45	Live Gemini moderation classification against the real provisioned key (D-08 fix)	Manual (real key, `server/src/ai/moderation.ts`)	A genuine, non-fallback `allow` / `flag` / `block` verdict, not a `null` that degrades to the manual queue	`classifyText("Great pickup, right on time, very profess` `"review")` returned `{"verdict":"allow","categories":` `[],"piiFound":false,"confidence":0.99,"reason":"The text` `legitimate review with no presence of abuse, harassment,` `PII."}` — a real classification
T-46	Gemini model-ID resilience: the candidate-list fallback skips a 404'd model instead of failing closed entirely	Manual (real key, direct HTTP probe of 6 candidate IDs)	At least one candidate in the list resolves	`gemini-2.5-flash` / `gemini-2.0-flash` → `404` ; `gemini-3.6-` `3.5-flash` / `gemini-3.7-flash` / `gemini-flash-latest` → `200` `gemini-3.6-flash` first and succeeds immediately
T-47	FR-27: `GET /requests/mine` exposes the right location to the right side, gated the same way as contact reveal	Integration	Collector always sees the household's pickup point (already shared at request creation); household sees no collector location before acceptance, then the real one after	Confirmed in `requests.test.ts` — `collector_lon/lat` are `n` match the collector's position post-accept; `household_lon/lat` p cases
T-48	**(defect)** `GET /reviews/mine` — an author can see their own review regardless of visibility,	Integration + manual (D-10, reported by the user)	Author sees the review immediately with an honest status (`pending` / `visible`); a stranger's "mine" list never contains someone else's review	Confirmed live: a fresh review submitted, then approved and relea a real double-blind pairing (~15s in production, well under the 2-n interval) — the mechanism works; the gap was that neither side ha that it was working

#	Test case	Layer	Expected result	Actual result
	distinct from `GET /users/:id/reviews` (visible-only)			
T-49	Double-blind release fires quickly once both sides have genuinely reviewed each other on the same request	System (live, production)	Both reviews become visible to each other shortly after the second side submits	Two fresh reviews submitted on the same accepted request in prod passed AI moderation within ~10s and were mutually visible withi sweep cycle (~15s observed)
T-50	FR-28: a household can cancel its own pending request (previously impossible — only accept/reject existed, both collector-only actions)	Integration	`200` , status becomes `cancelled` with `cancelled_by:'household'` ; a second cancel attempt is `409`	Confirmed in `requests.test.ts`
T-51	FR-28: a collector can cancel an accepted request; a stranger cannot cancel a request they're not part of	Integration	Collector cancel `200` ; stranger cancel `409` (conditional update matches zero rows, same idempotency pattern as accept/reject)	Confirmed
T-52	**(superseded by T-62/T-63, TD-17)** FR-29: a collector's live position reaching the pickup radius marks the request arrived automatically, without a button	Integration	*(removed — arrival is now button-triggered, not heartbeat-triggered; see T-62)*	The old heartbeat-auto-arrival test was rewritten rather than kept behaviour, since the mechanism it asserted on (`checkArrivals()` `presence/routes.ts`) no longer exists
T-53	FR-29: once arrived, cancel is no longer offered — there's nothing left to back out of	Integration	Cancel attempt after `arrived_at` is set returns `409`	Confirmed in the rewritten `requests.test.ts` (now via `POST /` rather than a heartbeat)
T-62	FR-29/TD-17: the Arrived button's server-side gate — `can_mark_arrived` on `GET /requests/mine` and `POST /requests/:id/arrived` both require the collector's live position to be within the configured radius	Integration	Far away: `can_mark_arrived:false` , `POST /:id/arrived` → `400` . At the pickup point: `can_mark_arrived:true` , `POST /:id/arrived` → `200` , `arrived_at` set, `status` stays `'accepted'` . Calling it again after arrival → `400`	Confirmed in `requests.test.ts` ("the Arrived button is only offe collector's live position is within the arrival radius...") — same `ST_` server-side both times, the client's tap is never trusted on its own
T-63	FR-29: only the request's own collector can mark it arrived — not the household, not a different collector	Integration	Household attempt → `403` (`requireRole("collector")`); a stranger (verified, online) collector on someone else's request → `400` (conditional update matches zero rows)	Confirmed in `requests.test.ts` ("only the collector on the requ arrived...")
T-64	FR-29a: confirming arrival sends an SMS to the household; a delivery failure never blocks the arrival itself from being recorded	Manual (real GiantSMS gateway) + code inspection	`arrived_at` is set unconditionally by the same `UPDATE` ; `sendSms()` is called afterwards, best-effort, with a `console.error` on failure but no thrown error	`POST /requests/:id/arrived` against a request created with a r number returned `200` regardless of gateway state during local tes configured in the test DB, arrival still recorded) — consistent with arrival/fail-safe-on-notify design
T-65	FR-30a: a request in History shows no live route map; a "View conversation" toggle reveals the chat on demand instead of by default	Component/manual (`HouseholdHome` / `CollectorHome` , `isHistory` prop)	No `RoutePanel` rendered when `isHistory` ; `RequestReviews` only mounts after the toggle is clicked, on both the household's and the collector's card	Confirmed by inspection of `HouseholdRequestCard` / `CollectorH` `isHistory` branch and manual click-through in the dev build
T-66	FR-30b: a History request's chat is frozen read-only — no rating form, no reply composer, even for a party still eligible by the backend	Component (`RequestReviews.tsx` `interactive` prop)	With `interactive={false}` , existing messages still render but neither `RatingComposer` nor `ReplyComposer` ever mounts	Confirmed by inspection — the `{interactive && (...)}` guard the prop, independent of `canReview` / `needsRating`
T-67	FR-33: the route map's zoom level tightens as the two parties get closer	Unit (`RoutePanel.tsx` `zoomForDistance()`)	`>3000m→13` , `>1500→14` , `>800→15` , `>400→16` , `>150→17` , else `18`	Verified by inspection of the threshold table against the function b step function with no gaps or overlaps across the tested boundarie
T-68	FR-34: tapping a request card's small route map opens a full-screen lightbox with a dimmed backdrop and an explicit close control; the small preview itself stays non-interactive so it's safe inside a tappable button	Manual (dev build, keyboard + pointer)	Click/tap opens `role="dialog"` overlay; clicking the backdrop or the close icon closes it; the small map does not pan/zoom under touch (Leaflet handlers disabled via `interactive={false}`)	Confirmed in the dev build; also satisfies accessibility lint (`S6847` by using a real `<button>` backdrop instead of a `div onClick`
T-69	FR-35: the collector Home map renders edge-to-edge, with the online/offline toggle floating on top of it	Manual (dev build, narrow viewport)	Map fills the full device width (no visible side gutter against the app shell's padding); the toggle button sits centred over the bottom of the map, not stacked above it	Layout/width confirmed at a 375px-wide viewport, but the button be briefly visible then hidden behind the map once tiles painted — initial pass only checked position, not whether it stayed visible afte
T-70	**(defect)** D-14: the "Add Borla to your home screen" label disappears once the app is actually installed, matching	Component/manual (`Profile.tsx` , `useInstallPrompt`)	With `display-mode: standalone` (or, on older iOS, `navigator.standalone`) true, neither the label nor the install button renders — only the "✓ Borla is installed on this device." line does	**Reported by the user** ("why am I seeing 'Add Borla to your hom when I installed the app") — traced to the label being a hardcoded never gated on install state at all

#	Test case	Layer	Expected result	Actual result
	`InstallButton` 's own already-correct state			
T-71	**(defect)** D-15: a `.map-overlay-btn` placed on top of any `.map-wrap` map stays visible once Leaflet's tiles/controls finish painting, instead of being buried by their internal z-index	Manual (dev build)	The floating "Go offline" button remains visible and clickable throughout, on first render and after tiles/markers load	**Reported by the user** ("the go offline button… glitches into view disappears") — traced to `.map-wrap` never establishing its own C context, letting Leaflet's panes (z-index up to 1000) escape into the stacking context and outrank the button's `z-index: 15` there
T-72	**(defect)** D-16: a fully-interactive map (collector Home map, `RoutePanel` lightbox) stays where a user manually pans/zooms it, instead of snapping back on the next live coordinate/zoom update	Manual (mobile device, real GPS movement)	After a manual drag or pinch-zoom, subsequent heartbeat-driven position updates no longer force the viewport back to centre; the live "you are here"/route markers still move to their correct positions regardless	**Reported by the user**, testing as a collector on a mobile device (map… is difficult; reset when you move") — traced to `Recenter` c `map.setView(...)` unconditionally on every centre/zoom prop c for a manual gesture to "win"
T-73	**(defect)** D-17: the topbar (logo, logout button) and tab bar clear a notched iPhone's status bar and home-indicator strip	Manual (iPhone 17)	Logo/logout button fully visible below the network/battery/time icons; tab labels clear of the home-indicator strip	**Reported by the user** ("on iphone 17, the logo and the logout bu under network and battery icons") — traced to `viewport-fit=cov` `translucent` drawing content under the status bar with no match `area-inset-*)` padding anywhere in the CSS. The topbar/tabbar fixed correctly; see T-74/T-75 for two follow-on regressions this sa elsewhere, caught on the same device immediately after
T-74	**(defect)** D-18: the tab bar's icons/labels stay correctly centred and full-height on a notched device, instead of being squeezed by the same safe-area padding that fixed T-73	Manual (iPhone 17)	Tab icons sit at the same vertical position/size as on a non-notched device; only the reserved strip below them (over the home indicator) grows	**Reported by the user** immediately after T-73 shipped ("the layo pane is now off. The icon don't sit right in the pane.") — traced to fixed `height: 64px` while gaining `padding-bottom: env(safe- bottom)` , which (under `box-sizing: border-box`) shrank the ic growing the box
T-75	**(defect)** D-19: the first-launch splash screen's "Get started" button stays on-screen on a notched device, instead of being pushed off the bottom by the taller topbar	Manual (iPhone 17, fresh session / logged out)	The full splash — language picker and "Get started" button included — fits within the viewport below the topbar with no scrolling needed	**Reported by the user** ("also get started is buried when app is op time") — traced to the splash's hardcoded `minHeight: calc(100v` guess at the topbar's height that stopped matching once T-73 mad on notched devices
T-76	**(defect)** D-20: switching between the Home and Profile tabs no longer visibly "jumps" — the topbar/tabbar chrome stays mounted across the navigation	Manual (mobile device)	Topbar and tab bar render once and stay stable in place; only the page content between them changes when tapping Home/Profile	**Reported by the user**, persisting even after the map-focused D- had already shipped — traced to every route wrapping its own sepa instance, so React Router fully remounted the chrome on every na
T-77	**(defect)** D-21: a socket-driven status banner (e.g. "The household cancelled this request.") clears itself after a few seconds, and can also be dismissed manually	Manual/component (`HouseholdHome` , `CollectorHome`)	The banner disappears on its own roughly 6 seconds after appearing, or immediately on tapping "Dismiss"	**Reported by the user** ("stuck after it appears") — traced to the no dismiss mechanism, automatic or manual, at all
T-78	**(defect)** D-22: "Borla is ready to work offline" shows once, ever, not on every app launch	Manual (reopen an installed PWA session twice)	The banner appears the first time the service worker finishes precaching; reopening the app afterwards does not show it again	**Reported by the user** ("keeps appearing") — traced to the dismi being in-memory component state, with nothing persisted across a worker registration on relaunch
T-79	FR-36: a household cannot send a second request to a collector while an earlier one is still pending or already accepted; a new request is allowed again once the earlier one resolves	Integration	Second attempt while pending/accepted → `409` ; a different household is unaffected; a fresh request succeeds once the earlier one is cancelled	Confirmed in `requests.test.ts` ("a household cannot send a se collector while one is already pending or accepted") — DB-enforce unique index, not just an application check, so it holds even under
T-80	FR-37: `GET /requests/mine` orders by whichever event most recently made a request current (arrival/response), not strictly by creation time	Integration	An older request resolved *after* a newer, still-pending one sorts above it	Confirmed in `requests.test.ts` ("orders by whichever event mo request current, not by when it was created")
T-81	FR-38: a household's Requests tab separates "Awaiting response" from "On the way", mirroring the collector's Incoming/On-the-way split	Manual/component (`HouseholdHome.tsx`)	Pending (requested/seen) requests render under their own heading, separate from accepted ones; the tab's badge count still reflects both combined	Confirmed by inspection of `pendingRequests` / `acceptedReques` section JSX

#	Test case	Layer	Expected result	Actual result
T-82	Investigation: how long an unanswered request takes to auto-cancel	Manual (production config check + code inspection)	A documented, admin-tunable duration, not a hidden or inconsistent one	`request_timeout_seconds` defaults to 90s (`server/src/config` confirmed still at its default (90) in production via `GET /admin/c` (`requestTimeoutSweep` , `jobs/workers.ts`) runs every 15s, so a flips to `timed_out` within 90–105s of being sent, depending on s defect found — reported to the user as a finding, and the exact num visible/changeable in Admin → Config ("Request timeout") per FR
T-54	FR-31: `GET /requests/:id/reviews` shows both directions of review in the context of the request they belong to	Integration	Author sees `mine` regardless of status; the subject sees `theirs` only once visible; a non-party gets `403`	Confirmed in `reviews.test.ts`
T-55	**(defect)** Removing a previously-visible review recomputes the subject's `rating_avg` / `rating_count` instead of leaving them stale	Integration (found while implementing FR-31, D-11)	Admin `POST /reviews/:id/remove` on a visible 5-star review drops the subject's `rating_count` back to 0, not left at the pre-removal value	**First implementation only updated the aggregate when th release sweep itself made a review visible — an admin rem afterwards never re-ran the computation** — see §4
T-56	`StatusChip` renders `'On the way'` / `'Arrived'` distinctly for an accepted request, and a label for `cancelled`	Component	Each status/arrived combination shows its own label	3/3 cases correct (`StatusChip.test.tsx`)
T-57	**(defect)** TD-15: a review now joins the shared thread the moment it clears moderation, instead of waiting for the other side to also review	Integration	A review approved via moderation appears in `GET /users/:id/reviews` immediately — no dependency on the other party's own review	Confirmed in `reviews.test.ts` — visible right after `status='v` second review on the request at all
T-58	TD-15: either party can reply to a review, more than once each	Integration	Both the review's author and its subject can each post multiple replies; a non-party gets `403`	3 successive replies from both sides confirmed in `reviews.test.` correctly rejected
T-59	TD-15: `GET /requests/:id/reviews` returns one identical shared thread to both parties, in order	Integration	Both callers see the same `messages` array (review + reply, chronological); each caller's own not-yet-visible submission only shows to them via `mine`	Confirmed — both sides' responses matched exactly once cleared; the author saw their own `mine.status='pending'`
T-60	**(defect)** D-13: an admin can manually approve a reply stuck awaiting moderation, not just a review	Integration	`POST /admin/replies/:id/approve` makes it `visible` ; a second attempt on the same reply is `404` (already resolved)	Confirmed in `admin.test.ts`
T-61	D-13: a Gemini call failure with a key configured is distinguished from "no key at all"	Manual (real key, direct `classifyText` call in isolation)	The exact text of a reply stuck in production's manual queue classifies successfully when called directly, supporting "transient failure," not "genuinely unmoderatable content"	`classifyText("Thank you so much, see you again!", "revi` returned a real `{"verdict":"allow","confidence":0.99,...}` job's context

4. Defects found during development

Defect	Where	Root cause	Corrective action
D-01	`server/src/utils/jwt.ts`	`jsonwebtoken` 's TypeScript types don't accept a plain `string` for `expiresIn` (expects its own `StringValue` literal union)	Cast through `jwt.SignOptions["expiresIn"]` explicitly rather than loosening the type globally
D-02	`server/src/modules/auth/routes.ts` (OTP verify)	An early draft wrapped user-creation in a transaction block that threw a sentinel error (`role_missing`) to short-circuit — but the `catch` swallowed it without ever inserting the user, so first-time signup silently returned an "unexpected signup state" error	Rewritten as a straight-line `if (!user) { ...INSERT... }` with no transaction/sentinel-error indirection; covered by T-02/T-08
D-03	`server/src/seed.ts`	First draft referenced a non-existent `verified_note` column on `collectors` (copy-paste artefact) wrapped in a silent `.catch()` fallback that masked the real error	Removed the phantom column and the `.catch()` swallow entirely; seed now fails loudly if it ever breaks again
D-04	`server/src/modules/reviews/routes.ts` (reply endpoint)	Only checked `review.subject_id === user.id` , not `review.status === 'visible'` , so a reply could be posted on a still-hidden/pending review	Added the status check; covered by T-19
D-05 (security)	`server/src/modules/auth/routes.ts` (`/auth/otp/request` , `/auth/otp/verify`)	Neither endpoint checked `users.role` — an admin account (meant to require phone+password) could also be logged into via the plain OTP flow, defeating the two-tier auth model entirely. Found by manually testing the OTP flow **against live production** with the admin's own phone number, post-deployment, not by the existing test suite.	Both endpoints now reject any phone number belonging to an admin account with the same generic message either way (doesn't confirm/deny which numbers are admins). Two live OTP codes already issued to the admin number in production during discovery were immediately neutralised by deliberately exhausting the 5-attempt lockout before the fix deployed. Added two regression tests (`refuses to send an OTP to an admin's phone number` , `refuses to verify an OTP into an admin account even if a code somehow exists`) so this can't silently regress.

Defect	Where	Root cause	Corrective action
D-06	`server/src/modules/auth/routes.ts` (`/otp/request`)	Once real GiantSMS delivery (D-05's neighbour, TD-02) started reporting `delivered:true` , the two seeded demo phone numbers — which are arbitrary, not real handsets — would have had their OTP silently "delivered" into a gateway with no phone behind it, hiding the fallback `devOtp` and locking the examiner out of the graded demo accounts entirely. Found by re-reading the deployment credentials file after wiring up real SMS, before it ever actually caused a lockout.	`DEMO_PHONES` / `DEMO_OTP` added to `server/src/seed.ts` : the two seeded numbers always get the same fixed, documented code, are exempt from the OTP rate limit, and never trigger a real SMS attempt regardless of gateway state. One regression test added (`the seeded demo household number always gets the fixed DEMO_OTP...` , 8 rapid requests all succeed and return the fixed code).
D-07	`server/src/modules/auth/routes.ts` (`phoneSchema` , both OTP endpoints)	Phone numbers were stored and looked up as the raw string the client sent, with no normalisation — `+233200000001` , `233200000001` , and `0200000001` were three different rows to Postgres, so an already-registered user who typed their number without the leading `+` looked brand new and was sent through the sign-up (role + name) path instead of logging straight in. **Found by the user testing the seeded household account (`233200000001` , no `+`) against the live app.**	Added `server/src/utils/phone.ts` (`normalizePhone`), wired into `phoneSchema` via Zod's `.transform()` so every route that accepts a phone number normalises it before it ever reaches a query or an insert. Four new unit tests (`phone.test.ts`) plus regression coverage in `auth.test.ts` confirming the same number in all three input forms resolves to one account.
D-08	`server/src/ai/moderation.ts` (`MODEL` constant)	The moderation pipeline was hardcoded to `gemini-2.5-flash` . Once the user provisioned a real `GEMINI_API_KEY` , live calls started returning `404 — this model is no longer available to new users` for that account's tier, which the fail-closed design (correctly) turned into every review silently landing in the manual queue instead of surfacing a visible error. **Found via production logs after the key was set.**	First fix (switching to a single hardcoded `gemini-flash-latest`) still couldn't be confirmed live. Root-caused properly by probing six candidate model IDs directly against the real key: `gemini-2.5-flash` / `gemini-2.0-flash` both 404, `gemini-3.6-flash` / `gemini-3.5-flash` / `gemini-3.7-flash` / `gemini-flash-latest` all work. Rewrote `classifyText` to try an ordered candidate list and cache whichever one succeeds per process, instead of betting on one guessed ID — confirmed working end-to-end locally (T-45/T-46).
D-09	`client/src/components/Avatar.tsx` , `client/src/styles/tokens.css`	Two related defects, both found by the §6 screenshot pass rather than by reading the code: (1) the `Avatar` component referenced `.avatar` / `.avatar.g` classes that had never actually been added to `tokens.css` , so initials rendered as bare unstyled text with no circular background; (2) the initials helper took the first character of the *last* whitespace-separated token without checking it started with a letter, so `"Ama (Osu)"` produced `"A("` .	Added the missing `.avatar` / `.avatar.g` rule block; filtered the initials helper to letter-led words only. Re-screenshotted to confirm both fixes.
D-10	`client/src/pages/Profile.tsx` , `client/src/components/ReviewForm.tsx`	The double-blind release mechanism itself was working correctly (confirmed by T-49), but nothing in the UI *said* so: `GET /users/:id/reviews` only ever returns `status='visible'` rows by design, and there was no way for the author of a review to see their own submission anywhere — not on Profile, not after the initial "submitted" moment. A one-sided review (the common case until the other party also reviews) looked indistinguishable from a silently-broken or lost one. **Reported by the user** ("why is approved reviews and given reviews not seen by either party") after testing the flow themselves.	Added `GET /reviews/mine` (any status, author-only) and a "Reviews I've given" section on Profile showing each review's real status (`Awaiting moderation` / `Approved — waiting on the other side…` / `Public` / etc.); reworded `ReviewForm` 's post-submit message to explain the hold instead of a bare "thanks".
D-11	`server/src/jobs/workers.ts` (`recomputeRatingAggregate`), `server/src/modules/admin/routes.ts`	`rating_avg` / `rating_count` were only ever recomputed from inside the review-release sweep, for the reviews *it* just released. Admin actions that change a review's status outside that sweep — removing a visible review directly, or resolving a moderation flag as "remove"/"clear" — updated `reviews.status` but never touched the subject's aggregate, leaving it stale (e.g. a removed 5-star review would keep inflating `rating_count` forever). **Found while adding FR-31's review-in-context view**, reasoning through every path that changes review visibility rather than just the sweep.	Exported `recomputeRatingAggregate` from `jobs/index.ts` ; both `/admin/reviews/:id/remove` and `/admin/moderation/flags/:id/resolve` now call it for the affected subject after updating status. Also hardened the function itself to reset to zero (not leave the previous value) when a subject ends up with no visible reviews at all. Regression test added (T-55).
D-12	`server/src/jobs/workers.ts` (`reviewReleaseSweep` , now removed), `server/src/modules/reviews/routes.ts`	The double-blind release model meant the two parties to the *same* request could see different content on the *same* request card at the same time — whichever side hadn't reviewed yet saw nothing, making the card look inconsistent or broken rather than "waiting on the other person". **Reported directly by the user** ("why can't both users see the same comments on the request card") together with a concrete ask: an open reply chain/chat, either party, AI-moderated.	This is TD-15, not a small patch: removed the reciprocal-release sweep entirely; a review or reply now goes `visible` the moment it individually clears moderation (same as replies always worked); `review_replies` ' one-per-review cap and subject-only restriction were both dropped so either party can reply, any number of times; `GET /requests/:id/reviews` now returns one identical `messages` thread to both callers. Documented as a deliberate trade-off (dropped collusion-resistance in exchange for consistency and a real chat) rather than a silent regression — see Technical_Debt_Plan.md TD-15. Tests: T-57–T-59.
D-13	`server/src/jobs/workers.ts` (`processModerate`), `server/src/modules/admin/routes.ts`	Found live in production while verifying D-12/TD-15: a transient Gemini failure (not "no key configured", an actual call failure — plausibly rate-limiting from this session's own heavy testing) left two genuine replies stuck in the manual queue indefinitely. Root cause: `processModerate` returned quietly on a null verdict instead of throwing, so the `moderate` queue's own `attempts:3` /exponential-backoff config (`jobs/queues.ts`) never engaged — a transient failure got exactly one attempt, forever. Compounding it: there was no admin "Approve" action for a stuck *reply* at all	`processModerate` now distinguishes "no key configured" (permanent, returns cleanly, no point retrying) from "the call itself failed" (throws, letting BullMQ's existing retry/backoff actually run) — confirmed locally that the exact stuck reply text classifies fine in isolation, supporting transient failure as the cause. Added `POST /admin/replies/:id/approve` , mirroring the review one, plus its "Approve" button in the admin UI for reply-type queue items (previously review-only). Regression test added (server/test/integration/admin.test.ts).

Defect	Where	Root cause	Corrective action
		(only reviews had one), so once stuck, a reply had zero recovery path.	
D-14	`client/src/pages/Profile.tsx`	The "Add Borla to your home screen for one-tap access." label next to `<InstallButton>` was a hardcoded sibling `<span>`, never gated on install state at all — unlike `InstallButton` itself, which correctly swaps to "✓ Borla is installed on this device." once installed. A user who had already installed the PWA kept seeing the "go install it" prompt text forever alongside the correct checkmark button underneath it. **Reported by the user**, who noticed the stale label after installing the app.	Profile now calls the same `useInstallPrompt()` hook `InstallButton` uses internally and only renders the label when `!installed`. While fixing it, also hardened `useInstallPrompt`'s own `installed` detection: it only checked the `(display-mode: standalone)` media query, which older iOS Safari never reflects for a manual "Add to Home Screen" — added a fallback check against `navigator.standalone` so an iPhone user who installed that way doesn't hit the same stale-label bug for a different reason.
D-15	`client/src/styles/app.css` (`.map-wrap`), `client/src/pages/CollectorHome.tsx` (Home tab map)	The floating "Go offline" button added on top of the enlarged Home map (FR-35) would flash into view for one frame, then disappear permanently. Root cause: Leaflet assigns its internal panes real `z-index` values (tile pane 200, popup pane 700, controls 1000) but never gives `.leaflet-container` itself a `z-index`, and neither did `.map-wrap` — a `position: relative` box with no `z-index` doesn't establish a CSS stacking context, so those panes escape all the way up to the page's *root* stacking context, where they were compared directly against the button's much lower `z-index: 15` and buried it the moment tiles finished painting. A related, smaller issue in the same code: the enlarged map's wrapping div and the `MapView` nested inside it were both given the identical `map-wrap hero full-bleed` classes, double-applying `full-bleed`'s negative edge margins and shifting the map slightly outside its own container. **Reported by the user** while this session's map-enlargement work was still fresh.	Added `isolation: isolate;` to the base `.map-wrap` rule — this seals every map's own internal stacking (however high Leaflet's z-index values go) inside its own box without changing how the box itself stacks against page siblings, so the fix applies to every map on the site, not just this one. Introduced a `.map-fill { height:100%; width:100% }` utility so a `MapView` nested inside an already-sized `.map-wrap` box only fills it, instead of re-declaring the box's own size/margin rules a second time. **Follow-up**: the user reported the lightbox close button was now fixed but the Home tab's go-offline button still glitched on the same device. Since both fixes shipped in the same deploy, the leading suspect is the installed PWA serving a stale cached bundle rather than a second distinct bug — `UpdatePrompt.tsx` deliberately never swaps content under an open tab (`registerType:'prompt'`, TD-04), so an already-installed instance keeps running whatever was cached until its "Update available" banner is accepted or the app is fully closed and reopened. Added defensive hardening regardless, cheap and safe either way: `isolation: isolate` directly on `.map-overlay-btn` and `.map-fill` too (not just the outer `.map-wrap`), and raised `.map-overlay-btn`'s z-index from 15 to 50 to match the same floating-control tier `UpdatePrompt`'s own banner already uses. Still awaiting confirmation from the user on a freshly-updated/reinstalled instance.
D-16	`client/src/components/MapView.tsx` (`Recenter`)	Every fully-interactive map (the collector's Home map, and any expanded `RoutePanel` lightbox) force-called `map.setView(...)` on *every* change to the live centre coordinates or the distance-based zoom — which happens every few seconds while a collector is online and roaming (position heartbeats) or a route's distance recomputes. On mobile this made the map effectively impossible to pan or pinch-zoom: any manual gesture was silently reverted within seconds by the next live update, snapping the view back to a re-centred position. **Reported by the user**, testing as a collector on a mobile device: "navigating the map… is difficult; reset when you move."	`Recenter` now tracks a `follow` flag, on by default: it auto-centres exactly as before *until* a real `dragstart` or user-initiated `zoomstart` fires, at which point it stops recentring for the life of that map instance — the "you are here"/route markers keep tracking live coordinates regardless (they're plain props, independent of the viewport-following logic), only the forced view-snapping stops. A `programmaticZoom` ref distinguishes the component's own `setView`-triggered zoom events from a genuine pinch/double-tap so the guard doesn't misfire on its own updates.
D-17	`client/index.html` (`viewport-fit=cover` + `apple-mobile-web-app-status-bar-style: black-translucent`), `client/src/styles/app.css` (`.topbar`, `.tabbar`)	On notched iPhones (reported on iPhone 17), the brand logo and the logout button rendered underneath the status bar's network/battery/time icons, and the bottom tab bar sat flush against the home-indicator strip. Root cause: the app deliberately draws edge-to-edge under the status bar (`viewport-fit=cover` + `black-translucent`, needed for the full-bleed map work, FR-35) but nothing in the CSS ever added the matching `env(safe-area-inset-*)` padding those two settings require — every other device without a notch happened to have zero inset, so the gap was invisible until tested on real notched hardware. **Reported by the user.**	Added `padding-top: calc(14px + env(safe-area-inset-top))` to `.topbar` and `padding-bottom: env(safe-area-inset-bottom)` to `.tabbar` (with `.content`'s bottom padding bumped to match the now-taller tab bar) — `env()` resolves to `0` on any device without a safe-area inset, so this is a no-op everywhere else.
D-18	`client/src/styles/app.css` (`.tabbar`)	D-17's own fix introduced a new bug on the exact devices it was meant to help: `.tabbar` kept its fixed `height: 64px` while gaining `padding-bottom: env(safe-area-inset-bottom)` — with the project's global `box-sizing: border-box`, that padding ate into the fixed height instead of growing the box, squeezing the tab icons/labels into a shorter content area on any notched device. **Reported by the user** ("the layout in the button pane is now off. The icon don't sit right in the pane.") immediately after D-17 shipped.	Changed `.tabbar`'s `height` to `calc(64px + env(safe-area-inset-bottom))` so the inset grows the box instead of shrinking the icon area — the icons' own 64px stays exactly as it was on every device; only the reserved strip below them grows on a notched one.
D-19	`client/src/pages/Login.tsx` (splash screen)	Also surfaced by D-17's own fix: the first-launch splash screen hardcoded `minHeight: calc(100vh - 64px)`, guessing `.topbar`'s height as a fixed 64px so the splash box would fill the rest of the viewport. Once `.topbar` grew by `env(safe-area-inset-top)` on a notched phone, the real topbar height exceeded that guess, so the splash's own box (still sized against the old assumption) overflowed the viewport and pushed the "Get started" button off the bottom of the screen on first open — the one moment every new user has no other way into the app. **Reported by the user** ("get started is buried when app is open for the first time").	Replaced the hardcoded `calc(100vh - 64px)` with `flex: 1` — `.app-shell` is already a flex column and `.content` already uses the same `flex: 1` approach elsewhere, so the splash now fills exactly whatever space is left below the topbar, however tall that topbar actually renders, with no magic number to fall out of sync again.

Defect	Where	Root cause	Corrective action
D-20	`client/src/App.tsx` (`Shell` , route tree)	A visible "jump" persisted when switching between the Home and Profile tabs even after every CSS-level safe-area/height fix (D-17–D-19) — because the actual root cause was never CSS at all. Every route wrapped its own separate `<Shell>{children}</Shell>` element, so React Router fully unmounted and rebuilt the entire topbar/tabbar chrome (and re-ran the safe-area `env()` calculations from scratch) on *every single navigation* between pages, not just on first load. **Reported by the user**, persisting after D-15/D-16's map-focused hardening pass had already shipped and been confirmed live for the *map* overlay specifically — the tab-switch jump was a distinct, still-open issue.	Restructured routing to a single shared layout route (`<Route element={<Shell/>}>` wrapping the page routes as children, rendered via `<Outlet/>`) — `Shell` now mounts exactly once for the whole authenticated session; navigating between Home/Requests/History/Profile only swaps the routed page content inside it, never the chrome around it. `wide` (admin's layout) is now derived from the current path instead of passed as a prop, since `Shell` no longer receives one per-route.
D-21	`client/src/pages/HouseholdHome.tsx` , `client/src/pages/CollectorHome.tsx` (`info` banner)	Every socket-driven status message ("The household cancelled this request.", "🎉 Your collector has arrived!", "Request sent.", etc.) stayed on screen forever once shown — there was no auto-dismiss and no manual close button at all, only ever replaced if some *other* event happened to set a new message first. **Reported by the user**: "'The household cancelled this request' and other, stuck after it appears."	Added a small `useAutoDismiss` hook (`client/src/hooks/useAutoDismiss.ts`) that clears the message after 6 seconds, plus an explicit "Dismiss" button on the banner itself for immediate control — applied identically in both `HouseholdHome` and `CollectorHome` , since both shared the exact same unbounded `info` state pattern.
D-22	`client/src/pwa/UpdatePrompt.tsx`	"Borla is ready to work offline" reappeared on every single app launch instead of once, ever. Root cause: the service worker re-registers (and re-fires `onOfflineReady`) each time the installed PWA is closed and reopened, and the previous dismissal (`setOfflineReady(false)`) was only ever in-memory component state — nothing persisted the fact that this purely informational, one-time message had already been shown. **Reported by the user**: "'Borla is ready to work offline' that keeps appearing."	Added a `localStorage` flag (`borla:offlineReadySeen`) set the first time the banner is genuinely earned; a value captured once at mount from a *previous* session's flag gates the render, so it still shows normally the first time it's actually true, but never again on subsequent launches.
D-23 (test infra)	`server/src/app.ts` (general `express-rate-limit` , 120/min per IP)	Not a product defect — found while adding the two new tests for FR-36/FR-37 (T-79/T-80): `requests.test.ts` had grown enough sequential Supertest calls within one shared `createApp()` instance to legitimately trip the app-wide 120-requests-per-minute safety net mid-file, failing an unrelated later test with a raw `429` instead of the JSON it expected. No test anywhere asserts on this middleware's exact threshold — it isn't the behaviour under test, just collateral from a growing suite sharing one rate-limited app instance.	Limit is now `config.isProd ? 120 : 100,000` — the real, enforced production value is completely untouched; only non-production runs (including every test file) are effectively uncapped, so continued suite growth can't trip this again.
D-24 (critical, production outage)	`server/migrations/004_request_guards.sql`	Production went down entirely — every deploy crash-looped on boot. `CREATE UNIQUE INDEX requests_active_pair_ux` (FR-36's own migration) fails outright if any existing row already violates it, and real usage (including this session's own heavy live-verification against the demo accounts) had already left more than one simultaneously-active request for the same household/collector pair. `server/src/db/migrate.ts` retries an unrecorded migration on every boot, so this failed, and failed again, on every subsequent start.	Added a one-time cleanup step ahead of the index creation: for any pair with more than one live request, keep the OLDEST (matching the guard's own future behaviour) and cancel the rest. Safe to edit 004 directly — by definition it never successfully committed in production, so it was never "applied" there. Verified empirically before shipping: reproduced the exact scenario against a disposable Postgres (two active requests, one pair, 2h apart) and confirmed the fixed migration + a full production-identical boot (`npm run build && node dist/index.js`) both succeed.
D-25 (critical, production outage)	`server/src/jobs/scheduler.ts`	Production went down a second time, immediately after D-24's fix — a *different* crash, `upsertJobScheduler("presence-sweep-scheduler")` throwing `WRONGTYPE` against Redis on every boot. Reproduced exactly locally by deliberately corrupting that one scheduler's own Redis key (hash → string) and getting an identical Lua-script/ `WRONGTYPE` error to production's log. The deeper cause, confirmed from a fuller log the user pasted: Upstash's free-tier Redis had hit its hard monthly command quota (`max requests limit exceeded. Limit: 500000, Usage: 500010`) — not a one-off corrupted key, but *every* Redis command being rejected for the rest of the billing period, most likely from this session's own very heavy live-testing.	`upsertJobScheduler` calls now retry a few times, then fall back to clearing that scheduler's own Redis state and recreating it. More importantly, `scheduleRepeatableJobs()` is now non-fatal to boot even if it still fails after retrying — the background sweeps matter, but "the entire API is unreachable" is a wildly disproportionate blast radius for one repeatable job's registration failing, which is exactly what took production down.
D-26 (critical)	`server/src/modules/presence/routes.ts` , `server/src/modules/broadcasts/routes.ts` , `server/src/redis/rateLimit.ts`	The Redis quota exhaustion behind D-25 turned out to break more than boot: live smoke-testing after D-25's fix found "Go online," broadcast creation, and (implicitly) OTP login all still capable of failing once Redis-dependent code ran mid-request — the architecture treats Redis as the sole source of truth for live geo-matching (Technical_Debt_Plan.md TD-05), with no fallback when it's unavailable. Fixing this surfaced a second, sharper problem: wrapping a Redis/BullMQ call in `.catch()` alone doesn't help if the call never rejects promptly — `bullRedis` 's required `maxRetriesPerRequest: null` , and ioredis's own retry/backoff internals more broadly, meant an unprotected call could hang for tens of seconds to over a minute rather than failing fast, found by explicitly timing a full request flow against a locally-simulated outage (47s before this fix; 18s after, both correct). A third, separate crash risk surfaced in the same investigation: ioredis can throw in ways that bypass	Added a `withTimeout()` helper (`server/src/utils/withTimeout.ts` , 2–5s per call) applied to every Redis/BullMQ call in a live request path, not just the boot-time scheduler. `GET /collectors/nearby` and `GET /pins/nearby` fall back to a direct Postgres/PostGIS geo query (`collectors.online` / `last_location` , `broadcasts.location`) when Redis's GEOSEARCH can't answer. `POST /presence` and `/presence/heartbeat` always write to Postgres regardless of whether the Redis mirror succeeded. `checkOtpRateLimit` / `checkNotifRateLimit` fall back to a small in-process counter (`server/src/utils/inMemoryRateLimit.ts`) instead of failing wide open — real, if per-instance-only, abuse protection for as long as an outage lasts, chosen over reimplementing BullMQ's queues in-memory (not worth the complexity for a temporary outage) and over an in-memory geo-matching fallback (Postgres is strictly better there — durable, and already

Defect	Where	Root cause	Corrective action
		promise rejection entirely (a subscribe-mode client — Socket.IO's Redis adapter uses one — flushing its offline queue once retries are exhausted), which a top-level `uncaughtException` / `unhandledRejection` handler (`server/src/index.ts`) now catches as defence in depth underneath every specific fix below, rather than as a substitute for them.	correct if this ever scales to multiple instances). The Socket.IO adapter's two duplicated Redis clients were also found missing their own `'error'` listeners (`realtime/socket.ts`) — added, independent of the uncaught-exception backstop.

All twenty-five were caught by testing immediately after (or, for D-05/D-06/D-07/D-08/D-10/D-12/ D-14 through D-22/D-24 through D-26, well after) the corresponding feature — eight by the automated suite (D-11 by reasoning through the code while building an unrelated feature, D-13 by directly observing stuck production queue items while manually verifying D-12), seventeen by deliberately exercising the live deployed app or reading its logs (D-05 by me re-testing production myself; D-07, D-10, D-12, D-14 through D-22, and D-24 through D-26 reported back by the user or caught by watching production directly; D-08 surfaced in production logs once the Gemini key went live), and D-09 by a scripted screenshot pass rather than by reading the code — direct evidence for why TD-10 (test depth) is listed as "scheduled," not "critical": the practice works, it just hasn't been extended to every corner of the app, including production behaviour, yet. D-10 and D-11 make a related point: a *correctly working* backend mechanism (the double-blind release sweep; the rating aggregate) can still fail users if either its state is never surfaced (D-10) or it's only kept correct along one of several paths that touch it (D-11) — functional correctness on the happy path and correctness everywhere the same data can change are different bars. D-12 is a step further still: the mechanism was working *exactly as designed*, and the design itself was the problem — no amount of additional testing against the original spec would have caught it, because the spec (double-blind release) was what the user was objecting to. D-13, found while manually re-verifying D-12's own fix in production, is a reminder that verification itself can surface new defects — the retry/backoff infrastructure existed since TD-05 but had never actually been exercised until real usage hit it. D-14 and D-15 are UI-layer versions of the same lesson as D-10/D-11: a component that individually works correctly (`InstallButton` 's own state; a Leaflet map rendering fine on its own) can still produce a visibly broken experience once placed next to something else that doesn't share its state (a stale hardcoded label) or contend with its stacking (an overlay button) — neither is caught by a unit test of the component in isolation, only by looking at the assembled screen. D-16 and D-17 are squarely device-class gaps: nothing in this project's own dev environment (desktop browsers, an Android emulator) has a notch or a GPS that streams position updates every few seconds while a person is physically walking with the page open — both defects are entirely invisible until a real person tests on real mobile hardware, which is exactly what surfaced them, well after both features individually passed their own review. D-18 and D-19 are the sharpest lesson of the batch: they are regressions introduced *by D-17's own fix*, each caught within one round-trip of shipping it because the same user immediately re-tested on the same real device — a reminder that a safe-area fix changes real, load-bearing pixel measurements (a fixed box height; a hardcoded viewport-height guess elsewhere in the codebase that had nothing to do with the fix itself) and every place that measurement was assumed constant needs re-checking, not just the one spot the original bug was reported in. D-20 is a reminder that a CSS-only fix can mask a symptom without curing the disease: D-15/D-16's map-focused hardening genuinely fixed the map overlay's own stacking, but the *general* Home↔Profile navigation jump the user kept seeing turned out to be an architectural issue one layer up (the whole chrome remounting per route) that no amount of further CSS tuning on the map itself would ever have touched — it needed the actual remounting to stop, not a better z-index. D-21 and D-22 are both instances of the same gap: a piece of transient UI state (`info` ; `offlineReady`) with a clear "this should eventually go away" intent, but no code that actually implemented that intent — correct on the happy path (showing the message) and silently wrong on the other half (never un-showing it). D-23 is listed separately, deliberately outside the count above — it's a test-harness capacity limit the suite's own growth ran into, not a defect in the product being tested, and is recorded here for the same reason everything else on this page is: a decision was made and it should be visible, not just quietly committed. D-24 through D-26 are a single incident told across its full arc, deliberately not compressed into one entry: a first fix (D-24) was real and necessary but not the actual cause of the outage it shipped alongside; a second fix (D-25) found and killed the actual crash, then led straight to its own root cause (an exhausted third-party Redis quota); and following that thread properly (D-26) surfaced two more layers — a whole class of live request paths with no fallback at all, and a promise-bypassing crash risk no amount of `.catch()` placement would ever have caught. Each layer was only visible once the one above it was fixed enough to reveal what was underneath — a reminder that "the crash stopped" and "the root cause is fixed" are not the same milestone, and that a production incident is worth following all the way down rather than stopping at the first fix that makes the symptom go away. Some defects are only found by putting the actual feature in front of the actual person it's for, and some are only found by watching your own fix operate for real.

5. Security testing

Check	Method	Result
Every mutating/reading-sensitive route requires a valid JWT	Automated (T-05, T-12, T-16, T-22) + manual `curl` without a token	Consistently `401` / `403`
SQL injection surface	Code review — every query in `server/src/modules/**` uses parameterised `$1..$n` placeholders, none concatenate user input into SQL	No string-built SQL found
OTP brute-force / spam	Code review + manual test + unit (T-43)	5-per-10-minutes request cap (now Redis-backed, `checkOtpRateLimit` , per-phone rather than per-IP — Technical_Debt_Plan.md TD-05) and a 5-attempt verify cap are enforced (`otp/routes.ts`)
Role escalation (collector calling household-only routes, etc.)	Automated (T-12) + manual `curl` across all role-gated routes	Consistently `403`
Contact information leakage pre-accept	Automated (T-14)	Phone fields verified absent from the JSON payload itself (not just hidden in the UI) before acceptance
Secrets handling	Code review	JWT secrets and the Gemini key are read from environment variables only, never committed (`.env` is git-ignored, `.env.example` has placeholders)
Admin account reachable via the weaker OTP path (bypassing phone+password)	Manual testing against the **live production** deployment	**Confirmed exploitable** (D-05) — fixed immediately, redeployed, and re-verified `400` on both endpoints for the admin's number

6. Usability testing

A design-fidelity pass against the approved `borla_UI_design.html` mockups: the brand mark (pin + marigold radar dot), the line-icon system, coloured initial avatars, and the signature animated "broadcast ring" were all re-implemented in the real app (not just approximated with emoji) and verified with real, scripted screenshots (Puppeteer driving headless Chrome against the running dev server, logged in as the seeded demo accounts) rather than eyeballing the code.

D-09 (found this way) — see §4 — covers both the missing avatar styling and the broken initials logic that the screenshot pass surfaced.

A second usability pass, prompted directly by user feedback rather than a screenshot diff: admin-facing copy had leaked internal spec references (`design §17` , `Technical Debt Plan TD-01/TD-02`) into user-visible text, and a live-ops legend spelled out colour names in prose ("gold = active pins, green = online collectors") instead of showing an actual colour swatch. Both are informational-only, so no defect ID or regression test was warranted — fixed directly in `AdminDashboard.tsx` / `Login.tsx` / `auth/routes.ts` and the new `.legend-dot` style (`tokens.css`), re-verified by re-reading the rendered JSX rather than a screenshot.

Beyond that: tap targets ≥44px (`.btn` padding), icon-first primary actions, a single primary action per screen, and colour contrast matching the approved palette (green/marigold/coral on a warm paper background) — confirmed manually in-browser at mobile viewport widths (390px, 414px) and desktop.

7. Performance testing

Out of scope at pilot scale by design (see SRS NFR discussion). Two checks performed: (1) the PostGIS `ST_DWithin` queries that Postgres still runs as the durable mirror use the `GIST` index created in `001_init.sql` (`EXPLAIN` on `broadcasts_loc_gix` confirmed an index scan, not a sequential scan, even against the small seeded dataset); (2) since Technical_Debt_Plan.md TD-05, the actual hot-path nearby-collector/nearby-pin lookups run against Redis `GEOSEARCH` (`server/src/redis/presence.ts`) instead of Postgres on every request — confirmed functionally correct (T-41) but not load-tested at real traffic volume.

8. What was NOT tested (honestly stated, ties to Technical_Debt_Plan.md TD-10)

Visual confirmation of a GiantSMS text actually arriving on a real handset (T-33 confirmed the gateway *accepts* the request in production; the seeded demo number is a placeholder, not a live phone someone was watching); a live Gemini moderation call *specifically against the deployed production process* (T-45/T-46 confirm the fixed code works end-to-end with the real key run locally — production just needs to be confirmed running the same code with the same env var, not re-derived from scratch); concurrent double-accept under real network race conditions (only sequential-call idempotency is proven); load/stress testing; cross-browser automated testing (manual only); the four `CollectorHome` / `HouseholdHome` / `AdminDashboard` / `Profile` React pages have no component tests yet, only the two smaller components (`StatusChip` , `ReviewForm`).